SJ Myers

4/27/2026

CS 470 Final Reflection

https://youtu.be/uPnlDDTGSX0

Experiences and Strengths:

I have gained a solid foundation in cloud computing and contemporary application development during this training. I'm getting closer to my career objective of becoming a software developer with an emphasis on cloud-based and scalable systems thanks to this experience. I developed practical skills that go beyond theory and are immediately transferable to industrial employment by working with real-world technologies and deploying apps in a cloud environment. Learning how to create and implement apps using Amazon Web Services (AWS) was one of the most beneficial parts of this course. I obtained practical experience with services like DynamoDB for database administration, API Gateway for API management, Lambda for serverless computing, and S3 for storage. I was able to comprehend how various parts of a cloud-based system communicate with one another because to these technologies. I also improved my abilities to handle data flow between services, diagnose API problems, and work with JSON.

One of my main advantages as a developer is my ability to solve problems, particularly when faced with technical difficulties like troubleshooting API connection errors or fixing cloud service configuration problems. Additionally, I have proven to be persistent while solving challenging issues, which is crucial in software development. My ability to swiftly comprehend and use new technologies, such as serverless architecture and cloud deployment methodologies, is another asset I developed. I feel ready to seek entry-level positions as a junior software developer, backend developer, or cloud developer based on the knowledge and expertise I acquired in this course. I have a strong foundation for working in settings that depend on distributed and scalable systems because of my experience to AWS and serverless technologies.

Planning for Growth:

As I advance as a developer, I would concentrate on utilizing serverless architecture and microservices to enhance the scalability and structure of programs. I would divide the system into smaller, independent

services rather than creating a single, monolithic application. For instance, different services might handle data processing, authentication, and API handling. This strategy increases adaptability, simplifies updates, and permits autonomous component scaling. I would rely on AWS Lambda's built-in features, which automatically adapt to the volume of incoming calls, to handle scalability. This guarantees that the program can manage high and low traffic levels without the need for human involvement. I would use AWS CloudWatch to build structured logging for problem handling and make sure that APIs provide insightful error messages. Maintaining system dependability and diagnosing problems would be simpler as a result.

Predicting costs is another crucial aspect of cloud computing. By examining anticipated usage trends, like the quantity of API queries or data storage requirements, I would calculate expenses. It is simpler to track consumption and modify resources appropriately thanks to AWS's invoicing and monitoring capabilities. If usage is tracked and optimized, serverless services' pay-per-use business model can keep expenses low. Depending on the use scenario, both serverless computing and containers have advantages. Because you only pay for what you use, serverless computing is typically more predictable and economical for smaller applications or workloads with fluctuating traffic. However, containers can provide more stable pricing on a scale, particularly for applications that are frequently and heavily used. Additionally, containers provide you with more control over your surroundings, which is advantageous for more complicated systems.

When preparing for future growth, cloud-based architecture has both benefits and drawbacks. Reduced infrastructure management is a significant benefit since cloud providers take care of a large portion of the underlying hardware and scaling. Reduced control over some setups and possible reliance on a particular cloud provider are drawbacks, though. Although they take extra setup and upkeep, containers help overcome some of these constraints by providing more control. Elasticity and pay-for-service pricing are two important cloud concepts that affect design choices. Elasticity guarantees steady performance during periods of high utilization by enabling systems to automatically scale resources up or down in response to demand. Cloud computing is more affordable than traditional infrastructure because pay-for-service pricing makes sure that expenses are only incurred when resources are actively being used.

All things considered, this course has given me the technical know-how and strategic insight required to create and implement contemporary cloud-based apps. I intend to keep honing these talents going forward by investigating more sophisticated cloud services, honing my system design skills, and developing a deeper understanding of distributed and scalable systems.